Ders 13 Genişletilmiş Notu: Sonlu Durum Makineleri: Tanım, Durum Tablosu ve Diyagramı, Seri Toplayıcı ve Tasarım Örnekleri

Ders 13 Genişletilmiş Notu: Sonlu Durum Makineleri: Tanım, Durum Tablosu ve Diyagramı, Seri Toplayıcı ve Tasarım Örnekleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Sonlu Durum Makinesi Nedir? Günlük Hayattan Örnekler
- 2. Bilgisayar Bilimindeki Kullanım Alanları
- 3. Sonlu Durum Makinesinin Biçimsel Tanımı
- 4. Durum Tablosundan Durum Diyagramına
- 5. Bir Girdi Dizisi İçin Çıktıyı Bulmak
- 6. Başlangıç ve Bitiş Durumları
- 7. Makine Ne Yapıyor? Sağa Kaydırma Örneği
- 8. Seri Toplayıcı: İki Bir Bitlik Sayıyı Toplamak
- 9. Art Arda Üç 1 Gelince Hata Veren Makine
- 10. Dört Karakterlik Değişken Adı Denetleyicisi
- 11. Konunun Özeti ve Alıştırma Önerileri
- 12. Ayrık Matematik Dersinden Ne Bekleniyor?
- Örnek Sorular

1. Sonlu Durum Makinesi Nedir? Günlük Hayattan Örnekler

Hoca bu konuyu, dönem içinde yetiştirilemediği için hızlıca anlatacağını söyleyerek başladı. Öğrencilerin konuyu lojik dersinde de gördüğünü düşündüğünü, bu yüzden çok ayrıntıya girmeyeceğini belirtti.

Sonlu durum makineleri (finite-state machines) ile bir olayı ya da problemi durum diyagramları (state diagrams) kullanarak modelleyebiliriz. Hoca, “en basit otomat” diye nitelediği ve günlük hayatta sürekli kullandığımız ama bir sonlu durum makinesi olduğunu fark etmediğimiz örneklerden başladı.

1.1 Turnike

Metrobüse ya da vapura binerken karşımıza bir turnike çıkar. Turnike başlangıçta kapalı durumdadır. Akbili bastığımız an turnike kapalı durumdan açık duruma geçer. Biz aradan geçeriz ve turnike yeniden kapanır.

Bu olayda iki durum vardır: turnikenin açık olduğu durum ve kapalı olduğu durum. Kapalıdan açığa geçmenin şartı jetonu atmak (karta basmak) olur. Geçiş tamamlanınca turnike kapanır ve kapalı duruma dönülür.

1.2 Asansör

Asansöre binip bir kata çıkacağımızı düşünelim. Kapıyı açıp asansöre girmek bir durumdur. Durum değiştirmek için ilgili katın düğmesine basarız. Hoca bu geçişleri diyagramdaki kenarlar gibi düşünmemizi söyledi. Gidilecek kata gelince kapıyı açıp dışarı çıkarız. Artık durum değişmiştir: asansörün içinden dışına çıktık. Bu olay, biri kapıyı açıp içeri girdiğinde ve gideceği katın düğmesine bastığında yeniden başlar.

> **Hocanın vurgusu:** Önemli olan, o olayın içinde kaç tane durum olduğunu bilebilmektir. Olay bunun üzerine kurulur.

2. Bilgisayar Bilimindeki Kullanım Alanları

Hoca, bilgisayar biliminde sonlu durum makinelerinin nerelerde kullanıldığına kendi verdiği doğal dil işleme dersinden örnek verdi.

Doğal dil işleme: Bir kelimenin Türkçeye ait olup olmadığını bulmak istiyoruz. Kelimenin kökü ve o köke eklenen ekler Türkçenin dil bilgisi kurallarına uyuyor mu? Bunun için kurallarımız vardır ve bu kuralların her biri bir durumdur. Kurallara uyuldukça durumlar arasında ilerlenir. Son duruma gelindiğinde kelime bitmiş ve ardından bir boşluk gelmişse, kelime tamamlanmış ve Türkçenin bütün kurallarına uymuş demektir. Durumların hepsi geçilmiş ve olay bitmiştir.

Ses tanıma ve görüntü işleme: Aynı yaklaşım ses tanımada da kullanılabilir. Görüntü işlemede amaç, bir renk pikselinden sonra hangi renk pikselinin gelebileceğini tayin etmek olabilir. Hocanın örneği gökyüzüydü. Gökyüzünde genelde mavi hakimdir, bir mavi pikselden sonra yine mavi piksel gelme olasılığı çok yüksektir. Gözümüzü yukarıdan aşağıya indirip birden bir ağaca, tabiata geçtiğimizde maviden başka bir renge geçilir. Bu geçişin olasılığı belli bir değere ulaşınca "gökyüzü bitti, farklı bir renge geçtim" diyebiliriz. Hoca, gerçekte yalnız mavi ve yeşil değil, mavinin tonları gibi daha çok durum olabileceğini ve her birine geçiş olasılığının farklı olabileceğini ekledi.

> **Ek bilgi (derste anlatılmadı):** Görüntü işleme örneğinde geçişlere olasılık bağlanıyor. Durumlar arası geçişlerin olasılıkla verildiği modeller büyük olasılıkla Markov zincirleri ve gizli Markov modelleri (hidden Markov models) adıyla anılır. Bu derste görülen sonlu durum makinelerinde ise geçişler kesindir (deterministik): bir durum ve bir girdi tek bir sonraki durumu belirler.

3. Sonlu Durum Makinesinin Biçimsel Tanımı

Bir çizgenin düğümlerden ve kenarlardan oluştuğunu söylediğimiz gibi, bir sonlu durum makinesi M de belirli bileşenlerden oluşur. Hoca bunu bir altılı (6-tuple) olarak verdi:

$$M = (S, I, O, f, g, s_0)$$

- I: girdi (input) sembollerinin kümesi.
- O: çıktı (output) sembollerinin kümesi.
- S: durumların (states) sonlu kümesi. Bu makinede kaç durum olduğunu gösterir.
- f ve g: durumlar arasındaki hareketi anlatan iki fonksiyon.
- s_0: başlangıç durumu (start state). Elimizde onlarca durum olabilir; hangisinden başladığımızı bu bileşen söyler.

Hoca f ve g'yi, bir duruma giriş ve bir durumdan çıkış olarak anlattı: "Bir state'den çıkış fonksiyonu" ve "dönüş fonksiyonu". Her iki fonksiyonun da girdisi, makinenin girdi sembolleridir.

> **Ders sonrası not:** Hoca f'yi "durumdan çıkış", g'yi "dönüş fonksiyonu" diye anlattı. Standart tanımda (Rosen) $f : S \times I \to S$ geçiş fonksiyonudur (transition function): bulunulan durum ve gelen girdiden sonraki durumu verir. $g : S \times I \to O$ çıktı fonksiyonudur (output function): bulunulan durum ve gelen girdiden üretilecek çıktıyı verir. Durum tablosunda her hücredeki "durum, çıktı" ikilisi tam olarak (f, g) değerleridir.

4. Durum Tablosundan Durum Diyagramına

Hoca bir örnek verdi. Olayın ne olduğu bilinmiyor ve şu an önemli değil; yalnızca olayı anlatan tablo verilmiş:

- Girdi kümesi $I = \{a, b\}$,
- Çıktı kümesi $O = \{0, 1\}$,
- Durum kümesi $S = \{s_0, s_1\}$,
- Başlangıç durumu s_0.

Durum	f: a	f: b	g: a	g: b
s_0	s_0	s_1	0	1
s_1	s_1	s_1	1	0

Tablo şöyle okunur:

- s_0'dayken a gelirse yine s_0'a dönülür ve 0 çıktısı üretilir.
- s_0'dayken b gelirse s_1'e geçilir, yani durum değişir. Bu geçiş 1 çıktısıyla olur.
- s_1'deyken a gelirse s_1'de kalınır ve 1 çıktısı üretilir.
- s_1'deyken b gelirse yine s_1'de kalınır, çıktı 0 olur.

Hoca ardından aynı tabloyu durum diyagramına çevirdi. İki durum iki daire olarak çizilir. s_0'da kendi üzerine dönen bir kenar $a/0$ ile, s_0'dan s_1'e giden kenar $b/1$ ile etiketlenir. s_1'de kendi üzerine dönen iki kenar vardır: $a/1$ ve $b/0$. Başlangıç durumu s_0'a bir "start" oku konur. Hoca kenar etiketinde girdi ile çıktı arasına bazen virgül ($a, 0$), bazen eğik çizgi ($a/0$) koyduğunu söyledi.

> **Hocanın vurgusu:** Durum makinesi bazen tablo olarak verilir ve diyagramı çizmeniz istenir. Bazen de diyagram verilir ve tablosu istenir. İkisini birbirinden çizebilmelisiniz.

Hoca, her zaman tablo çizmek gerekmediğini de ekledi. Olay çok karmaşıksa hata yapmamak için tablo çizer; çok karmaşık değilse doğrudan durum diyagramı üzerinden çözer.

5. Bir Girdi Dizisi İçin Çıktıyı Bulmak

Sonraki örnekte beş durumlu bir makine verilmişti. Başlangıç durumu s_0'dır. Hangi girdi geldiğinde hangi duruma gidileceği ve hangi çıktının üretileceği diyagramda gösterilmişti. Hoca önce diyagramın nasıl okunacağını gösterdi: s_1'deyken 0 girdisiyle s_1'e dönülür, 1 girdisiyle s_2'ye gidilir ve 1 çıktısı üretilir.

Soru: Bu makineye `101011` girdisi verilirse hangi çıktı oluşur? İzleme şöyledir:

Adım	Bulunulan durum	Girdi	Sonraki durum	Çıktı
1	s_0	1	s_3	0
2	s_3	0	s_1	0
3	s_1	1	s_2	1
4	s_2	0	s_3	0
5	s_3	1	s_0	0
6	s_0	1	s_3	0

Çıktı dizisi `001000` olur. Hoca bu makinenin ne işlem yaptığının şu an önemli olmadığını, amacın çıktıyı bulmayı göstermek olduğunu söyledi. İzleme Python ile kontrol edildi; aynı çıktı elde edildi ve makine s_3'te bitti.

> **Ek bilgi (derste anlatılmadı):** Bu beş durumlu makine ve `101011` girdisi büyük olasılıkla Rosen'ın *Discrete Mathematics and Its Applications* kitabının sonlu durum makineleri bölümündeki örnekten alınmıştır. Kitapta bu tür çıktılı makinelere Mealy makinesi denir; çıktı, durum ile girdinin birlikte belirlediği geçişe bağlıdır.

6. Başlangıç ve Bitiş Durumları

Hoca, öğrencilerin şunu sorabileceğini söyledi: "Bu makinenin bir başlangıcı var, peki bir bitiş durumu yok mu?" Bunun için iki durum vardır:

- Elimdeki girdinin tamamı bitene kadar durumlar üzerinde ilerlemeye devam ederim; ayrı bir bitiş durumu yoktur.
- Ya da bir durumu iç içe iki halka (çift daire) ile gösteririm. Bu, o durumun bir bitiş durumu (final state) olduğu anlamına gelir.

Neden tek başlangıç, birden fazla bitiş? Başlangıç durumu tektir. Bitiş durumu ise bir tane de olabilir, n tane de olabilir; bu bir kural değildir. Hoca bunu az önceki izleme üzerinden açıkladı. `101011` girdisinde makine s_3'te bitti. Son biti koymasaydık başka bir durumda, bir önceki biti de koymasaydık yine başka bir durumda bitecektik. Gelecek girdinin boyunu bilmediğimiz için bu örnekte bütün durumlar bitiş durumu olabilir.

Bitiş durumunun belli olduğu örnek: şifre. Bir yere üye olurken şifre girmemiz istenir ve şifrenin en az 6 karakter olması gerekir. Bu durumda en kötü ihtimalle 6 durum geçildikten sonra ilk bitiş durumuna gelinmelidir. Şifre 7 ya da 8 karakter de olabilir. Üst sınır 10 karakter ise 10 durumlu bir makine düşünülür ve 6. durumdan sonraki bütün durumlar bitiş durumu olmalıdır.

> **Ders sonrası not:** Rosen'daki çıktılı sonlu durum makinelerinde (Mealy makinesi) bitiş durumu yoktur; makine her girdiye bir çıktı üretir. Bitiş (kabul) durumları, bir dizgiyi kabul eden ya da reddeden, çıktısız sonlu durum otomatlarına (finite-state automata) aittir. Hocanın şifre ve değişken adı örnekleri bu ikinci türe girer.

7. Makine Ne Yapıyor? Sağa Kaydırma Örneği

Sıradaki soru şuydu: "Aşağıda durum diyagramı verilen makine hangi işlemi gerçekleştirmektedir?" Makinenin üç durumu vardır (S_0, S_1, S_2), başlangıç S_0'dır ve bitiş durumu çizilmemiştir.

Hoca bitiş durumu çizilmemişse bunu "sürekli akan bir veri" gibi düşünmemizi söyledi. Verinin bittiği yerde olay biter; ya da girdinin boyunu bilmediğimiz için bütün durumları bitiş durumu yapabiliriz.

Bir makinenin ne yaptığını anlamak için örnek girdiler vermemiz gerekir. Hoca girdi olarak $12 = 1100_2$ sayısını aldı. Hocanın izlemelerinden makinenin geçişleri şöyle kurulabilir (tablo izlemelerden çıkarıldı, ara adım eklendi):

Durum	0 gelirse	1 gelirse
S_0	S_2, 0	S_1, 0
S_1	S_2, 1	S_1, 1
S_2	S_2, 0	S_1, 0

7.1 Önce sağdan okuma

Hoca önce girdiyi sağdaki bitten başlayarak verdi: 0, 0, 1, 1.

- S_0'da 0 geldi: S_2'ye geçildi, çıktı 0.
- S_2'de 0 geldi: S_2'de kalındı, çıktı 0.
- S_2'de 1 geldi: S_1'e geçildi, çıktı 0.
- S_1'de 1 geldi: S_1'de kalındı, çıktı 1.

Çıktı bitleri, okunan bitlerin yerine yazılınca $1000_2 = 8$ elde edildi. Hoca "12 iken 8 oldu" dedi.

7.2 Soldan okuma

Sonra aynı sayıyı soldaki (en anlamlı) bitten başlayarak verdi: 1, 1, 0, 0.

Adım	Durum	Girdi	Sonraki durum	Çıktı
1	S_0	1	S_1	0
2	S_1	1	S_1	1
3	S_1	0	S_2	1
4	S_2	0	S_2	0

Çıktı $0110_2 = 6$ oldu. Hoca bu çıktıyı yeniden makineye verdi: 0110 girdisi $0011_2 = 3$ çıktısını üretti. Bir kez daha, 0011 verilince:

- S_0'da 0: S_2, çıktı 0.
- S_2'de 0: S_2, çıktı 0.
- S_2'de 1: S_1, çıktı 0.
- S_1'de 1: S_1, çıktı 1.

Çıktı $0001_2 = 1$ oldu. 3'ün yarısı 1,5'tir; makine tam sayı işlemi yaptığı için 1 verir. Sonuç olarak makine $12 \to 6 \to 3 \to 1$ diye sayıyı her seferinde ikiye böler. İkiye bölme, ikili gösterimde bir bit sağa kaydırmadır (right shift).

> **Hocanın vurgusu:** Girdinin soldan mı sağdan mı okunduğu sonucu değiştirir. Makineyi tasarlarken ya da kullanırken bu bilgiyi vermek gerekir. Bu makine, ilk önce soldaki bit verildiğinde sayıyı ikiye böler.
>
> **Ders sonrası not:** Kısa notta "sağdan okumanın sonucu 8 doğrulanamadı" deniyordu. Yukarıdaki tablo hocanın izlemelerinden kurulunca 8 sonucu tutarlı çıkıyor. Makine aslında her adımda bir önceki girdi bitini çıktı olarak verir (ilk çıktı 0). Soldan okununca bu, sayıyı bir bit sağa kaydırır ($\lfloor n/2 \rfloor$). Sağdan okununca ve çıktılar aynı yerlere yazılınca sayı bir bit sola kayar ve 4 bite sığmayan bit atılır: $1100_2 \to 1000_2$, yani $(2 \cdot 12) \bmod 16 = 8$. Hoca bu durumu "sağa bir kez shift yapmışım" diye yorumladı. Python ile bütün 4 bitlik sayılar için soldan okumanın $\lfloor n/2 \rfloor$, sağdan okumanın $(2n) \bmod 16$ verdiği doğrulandı.

8. Seri Toplayıcı: İki Bir Bitlik Sayıyı Toplamak

Hoca "daha anlamlı şeyler yapalım" diyerek bir tasarım problemine geçti: İki tane bir bitlik sayıyı toplayan bir sonlu durum makinesi tasarlayın.

8.1 Girdiler, çıktılar ve durum sayısı

İki bir bitlik sayı toplanırken dört olası girdi vardır: 00, 01, 10, 11. Çıktılar yine 0 ya da 1'dir.

- $0 + 0 = 0$, elde 0.
- $0 + 1 = 1$, elde 0.
- $1 + 0 = 1$, elde 0.
- $1 + 1 = 0$, elde 1. İkili tabanda $1 + 1 = 2$ olur; 2 çıkarılır ve elde 1 kalır.

Kaç duruma ihtiyacımız var? Toplama işleminde ya elde oluşur ya oluşmaz. Bu yüzden iki durum yeterlidir:

- s_0: elde yok (başlangıç durumu),
- s_1: elde var.

> **Hocanın vurgusu:** Durum sayısına, "bu problemi çözmek için kaç durum gerekiyor?" sorusuyla karar verilir. Toplamada elde var ya da yok; çıkarmada borç alındı ya da alınmadı.

8.2 Elde yokken (s_0)

- 00 gelirse toplam 0, elde oluşmaz: s_0'da kalınır, çıktı 0.
- 01 gelirse toplam 1, elde yok: s_0'da kalınır, çıktı 1.
- 10 gelirse toplam 1, elde yok: s_0'da kalınır, çıktı 1.
- 11 gelirse toplam 2: çıktı 0, elde 1 oluşur. Elde oluştuğu için eldeli duruma, s_1'e geçilir.

8.3 Elde varken (s_1)

Bu durumda gelen iki bitin toplamına eldedeki 1 de eklenir.

- 01 gelirse $0+1+1=2$: çıktı 0, elde yine 1. s_1'de kalınır.
- 10 gelirse $1+0+1=2$: çıktı 0, elde yine 1. s_1'de kalınır.
- 11 gelirse $1+1+1=3$: bir elde çıkarılınca geriye 1 kalır. Çıktı 1, elde yine 1. s_1'de kalınır.
- 00 gelirse $0+0+1=1$: çıktı 1, elde kalmaz. Eldenin olmadığı duruma, s_0'a dönülür.

8.4 Durum tablosu

Durum	00	01	10	11
s_0 (elde yok)	s_0, 0	s_0, 1	s_0, 1	s_1, 0
s_1 (elde var)	s_0, 1	s_1, 0	s_1, 0	s_1, 1

Hoca kuralı şöyle özetledi: Girdiye karşılık bir çıktı üretilir. Elde oluşuyorsa eldeli duruma geçilir (ya da orada kalınır); elde yoksa eldesiz durumda kalınır.

Hoca, diyagramdan tabloyu da rahatça çizebileceğimizi, ama tablo çizmenin kendisine zahmetli geldiğini söyledi. Lojik dersinde tabloyla gösterilmiş olabileceğini, bunun o dersin içeriğinden kaynaklandığını belirtti. Kendisine durum diyagramı üzerinde, akış diyagramı çizer gibi çalışmak daha kolay geliyor; karar öğrencinin.

Tablo Python ile kontrol edildi: bitler en düşük basamaktan başlanarak verildiğinde, bütün 4 bitlik sayı çiftleri için doğru toplam elde edildi.

> **Ek bilgi (derste anlatılmadı):** Bu makineye "seri toplayıcı" (serial adder) denmesinin nedeni, çok bitli iki sayının bitlerinin sırayla, en düşük basamaktan başlanarak verilebilmesidir. Elde bir sonraki basamağa durum olarak taşındığı için aynı iki durumlu makine istenen uzunlukta sayıları toplar. Tek koşul, en sonda oluşabilecek eldeyi göstermek için girdiye bir 00 çifti daha eklemektir.

9. Art Arda Üç 1 Gelince Hata Veren Makine

Sıradaki tasarım problemi: Gönderilen mesajın içinde art arda üç tane 1 varsa, dışarıya hata mesajı olarak 1 değerini veren bir sonlu durum makinesi tasarlayın.

9.1 Problemin anlamı

Hoca tahtada birkaç mesaj yazdı. Art arda üç 1 içermeyen mesajlar için hata verilmez. Art arda üç 1 içeren bir mesaj hatalıdır ve dışarıya 1 verilir. Art arda beşten fazla 1 içeren bir mesajda ise üst üste binen her üçlü için ayrı bir hata mesajı verilebilir.

Hoca burada iki farklı soru olabileceğini belirtti:

- İlk hata mesajı alındıktan sonra işlem sonlandırılabilir. Bu durumda oraya bir bitiş durumu konur; üç tane 1 yakalanınca "tamam, gerisine bakma" denir.
- "Kaç tane art arda üçlü var?" denirse, her art arda üç 1 görüldüğünde dışarıya 1 verilir ve bu 1'ler sayılır. Bu başka bir sorunun cevabıdır.

9.2 Tasarım adım adım

Hoca makineyi kabaca çizerek kurdu. Amaç 1'leri saymaktır.

- s_0'dayken 0 gelirse kendi üzerine döner. Kaç tane 0 gelirse gelsin orada kalır.
- 1 gelirse durum değiştirilmelidir, çünkü bir 1 sayıldı. Dışarıya 0 verilir ve s_1'e geçilir.
- s_1'de 0 gelirse yeniden s_0'a dönülür ve dışarıya hata verilmez. 1'in arkasından 0 geldiği için artık üç 1 sayılamaz; 1'ler baştan sayılmalıdır.
- s_1'de ikinci 1 gelirse s_2'ye geçilir. Hâlâ dışarıya hata verilmez, çıktı 0'dır.
- s_2'de 0 gelirse iki 1'in hiçbir anlamı kalmaz; s_0'a dönülür ve 1'ler yeniden sayılır.

- s_2'de üçüncü 1 gelirse dışarıya hata mesajı, yani 1 verilir.

Hoca ilk çizimde üçüncü 1 için ayrı bir s_3 durumu açtı ve "belki şu an fazla yapacağım, bunu sileceğiz" dedi. Sonra şunu düşündü: Üç 1'den sonra 0 gelirse başa dönülmeli. Peki dördüncü bir 1 gelirse? Önündeki iki 1 ile birlikte yine art arda üç 1 olur ve yine hata verilmelidir. Bu yüzden s_2'de 1 gelince aynı durumda kalınıp 1 çıktısı verilir; bu kendi üzerine dönen kenar, 0 gelene kadar her yeni 1 için hata üretir. Fazladan çizilen durum silindi ve üç durum yeterli oldu.

Durum	0 gelirse	1 gelirse
s_0 (1 sayılmadı)	s_0, 0	s_1, 0
s_1 (bir 1)	s_0, 0	s_2, 0
s_2 (iki ya da daha çok 1)	s_0, 0	s_2, 1

Özetle: Her 0'da başa dönülür ve sayılan 1'ler unutulur. Birinci ve ikinci 1'de hata verilmez. Üçüncü 1'de hata verilir. Üç 1'den sonra 0 gelirse başa dönülür. Dördüncü 1 gelirse önündeki iki 1 de kullanılarak yine üç 1 olur, yine hata verilir ve aynı durumda kalınır.

Python ile rastgele dizilerde, makinenin çıktısının "son üç bit 111 mi?" sorusunun cevabıyla aynı olduğu kontrol edildi. Örneğin art arda beş 1 içeren `0111110` için çıktı `0001110` olur: üst üste binen üç üçlü için üç hata.

> **Hocanın vurgusu:** Bu akan bir veriyse, yani veri sürekli akıyorsa bitiş durumu oluşturulmaz. "İlk gördüğün hatada sistemden çık" denirse, s_2'den 1 ile çıkan kenar bir bitiş durumuna bağlanır ve ilk hatada sistemden çıkılır.

10. Dört Karakterlik Değişken Adı Denetleyicisi

Bu noktada hocanın tahta olarak kullandığı ekranda bir aksaklık oldu; hoca bir slayt açıp son örneği onun üzerinde anlattı.

10.1 Kurallar

Bir değişkene isim vermenin kuralları şöyle olsun:

- Değişken adı kesinlikle 4 karakter uzunluğundadır.
- 1., 2., 3. ve 4. karakterler büyük harf (A-Z) ya da küçük harf (a-z) olabilir.
- 0-9 arasındaki rakamlar yalnız 2., 3. ve 4. karakterde kullanılabilir. İlk karakter rakam olamaz.
- Bunların dışındaki karakterler (Ü, Ç gibi harfler, noktalı karakterler) Σ ile gösterilir. Böyle bir karakter gelirse sistem onu kabul etmez ve o karakterin yeniden girilmesini ister.

Örneğin `1AB4` geçerli bir ad olamaz, çünkü ilk karakter rakamdır.

10.2 Tasarım

- S_0 **(başlangıç):** İlk girilen karakter 0-9 arasındaysa kabul edilmez, S_0'da beklenir. Σ gelirse de kabul edilmez, yine S_0'da beklenir. A-Z ya da a-z arasında bir harf gelirse S_1'e geçilir.
- S_1**:** Yalnız Σ gelirse karakter beğenilmez ve yeniden girilmesi istenir (S_1'de beklenir). A-Z, a-z ya da 0-9 arasında bir karakter gelirse ikinci karakter kabul edilir ve S_2'ye geçilir.
- S_2**:** Yine Σ gelirse olduğu yerde durulur; A-Z, a-z ya da 0-9 gelirse S_3'e geçilir.
- S_3**:** Σ gelirse beklenir. Son karakter A-Z, a-z ya da 0-9 ise S_4'e geçilir.
- S_4 **(bitiş):** Değişken 4 karakter olacaktı; dört karakter tamamlandı. S_4 bitiş durumudur ve başka girdi kabul etmez.

Hatada baştan başlama: Sistem "her yanlış karakterde baştan gireceksin" derse, Σ kenarları kendi üzerine dönmez, hepsi S_0'a bağlanır. Hoca bunun, problemin nasıl verildiğine bağlı olduğunu söyledi.

Ders sonrası not: Hoca bu örnekte birkaç kez “epsilon” dedi, sonra düzeltip “sigma” demek istediğini söyledi. Kastedilen, geçersiz karakterlerin kümesi Σ’dır. Otomatlar kuramında ε genellikle boş dizgiyi gösterir; ikisi karıştırılmamalıdır.

10.3 Bu makinenin çıktısı var mı?

Hoca, “Hocam, bunların çıktısı yok” denebileceğini söyledi. Bu makine ayrı bir çıktı üretmiyor. Hocaya göre burada çıktı, ekrana basılan karakterin kabul edilmesidir: örneğin A-Z girdisi girildiyse çıktı A-Z’den herhangi biri, a-z girildiyse a-z’den herhangi biri diye düşünülebilir.

Değişken uzunluk: Problem “değişken en az 3, en çok 4 karakter olabilir” deseydi, 3. karakterden sonra sistemi durdurabilmek için o durum da bitiş durumu yapılırdı. Başka bir şey girilmiyorsa çıkılır, girilmeye devam ediliyorsa bir sonraki karakter alınır.

11. Konunun Özeti ve Alıştırma Önerileri

Hoca konuyu şöyle bağladı: Sonlu durum makinelerinde bir olay vardır. Olayın kaç durumla kurulabileceğine karar verilir. Durumlar arasındaki geçişler, olayı modellerken elimizdeki şartlardır. Akan bir veri varsa bitiş durumu yoktur; durma noktası belliyse bitiş durumları vardır.

Hoca kendi kendine alıştırma yapmak için şu soruları önerdi:

- Sağa kaydırmayı gösterdim; sola kaydırmayı siz deneyin.
- İki tane bir bitlik sayıyı topladım; iki tane iki bitlik sayıyı nasıl toplarsınız?
- Toplamayı yaptım; çıkarma işlemini nasıl yaparsınız?

Hoca bunlarla ilgili internette de çok sayıda örnek bulunduğunu söyledi. Bu alıştırmaların çözümleri aşağıdaki “Örnek Sorular” bölümündedir.

12. Ayrık Matematik Dersinden Ne Bekleniyor?

Dersin sonunda bir öğrenci genel bir soru sordu.

Soru: Bu ders diğer derslerden farklı ve çok kapsamlı; neredeyse her konu bir dönemlik. Konuları şu anda anlıyorum ama bir süre sonra çok azı aklımda kalacak. Bu dersten bizden ne bekleniyor, bu dersi neden alıyoruz?

Cevap: Ayrık matematik, dünyadaki bütün üniversitelerde ilgili mühendislik bölümlerinde birinci ya da ikinci sınıfta okutulan bir derstir. İçeriği bölüme göre değişir: elektrik mühendisliğinde başka konulara öncelik verilir. Bilgisayar biliminde amaç, öğrencinin 3. ve 4. sınıfta alacağı derslerin terminolojisini ve mantığını önceden tanımasıdır. Bu derste lojiğe, veri yapılarına, algoritmalara, derleyici tasarımına ve hesaplama kuramına bir giriş yapıldı. Seçmeli derslerde (ör. doğal dil işleme, veri madenciliği) karşılaşılacak algoritmaların içinde ağaçlar ve kodlamalar vardır; bunlara da ön bilgi verildi.

Hoca bunu bir benzetmeyle anlattı: Hayatında hiç televizyon görmemiş birini televizyonun karşısına oturtsanız, bir düğmeye ya da kumandaya basması gerektiğini bilmez, cihaza tuhaf tuhaf bakar. Ama daha önce televizyona benzeyen, düğmeyle açılan bir şey görmüşse bir kumanda arar ya da elini kenarlarında gezdirerek bir düğme arar. Bu ders de böyle bir ön bilgi verir. Dönem sonunda “hiçbir şey hatırlamıyorum” denebilir, ama o ilgili derse girildiğinde “şöyle bir şeyden bahsedilmişti” diye hatırlanır. Bu, konuları daha hızlı algılamayı ve daha kolay ilişki kurmayı sağlar.

Hoca bu dönem derse yeni konular eklendiğini de belirtti. Daha önceki yıllarda algoritmalar bu derste anlatılmıyordu, çünkü bölümde ayrı ve ayrıntılı bir algoritma dersi var. Çevrimiçi eğitimde tahta kullanılmadığı ve dersler daha hızlı aktığı için araya başka konular kondu.

Hocaya göre ayrık matematik bilgisayar biliminin olmazsa olmazıdır: veri yapıları, matematiksel ifadeler ve bir olayın matematiksel olarak modellenmesi. Örneğin her gün geçilen turnike de bir sonlu durum makinesidir. Bir şey açılır ya da kapanır ve bunun sebepleri vardır. Hoca, bir dersten geçmenin bile bir sonlu durum makinesi gibi düşünülebileceğini söyledi: belli şartlar sağlandığında bir sonraki duruma geçilir. Hayatımızdaki her şeyin arkasında bir matematik vardır.

Hoca dersten iki somut örnek hatırlattı:

- **Alt kümeleri bulmak:** İlk haftalarda, beş değişkenle yazılabilen bütün kombinasyonları yazma örneği görülmüştü; alt kümeleri bulmak gibi. İki değişken varsa ya ikisi de yoktur, ya ikisi de vardır, ya biri vardır biri yoktur. Bunun kodunu doğrudan düşünerek yazmak zordur. Ama ikili sayılar bu soruyla birebir örtüşür: 0'dan $2^n - 1$'e kadar sayıların ikili gösterimi bütün alt kümeleri verir. Bu derste görülen ayrık bakış açısıyla, belki 30 satır sürecek bir kod tek satırla yazılabilir.
- **Ağacı dizide tutmak:** Bir ağaç üzerinde arama yapmak şekil üzerinde kolaydır, ama kodu nasıl yazılır? Ağaç düğümlerine adres numaraları verildi ve bunlar dizinin indisleri yapıldı. Ağaçta sola gitmek indisi ikiyle çarpmak, sağa gitmek ikiyle çarpıp bir eklemek oldu: i indisli düğümün sol çocuğu $2i$, sağ çocuğu $2i + 1$ indisindedir.

  Hocanın vurgusu: Bu dersin vermek istediği şey bakış açısıdır: problemlere daha geniş bakmak ve görülen şeyleri problem çözmede nasıl kullanabileceğimizi görebilmek.

Örnek Sorular

Soru 1 (derste sözlü, 00:13:48)

Başlangıç durumu s_0 olan beş durumlu makineye `101011` girdisi verilirse çıktı ne olur?

Çözüm: İzleme $s_0 \xrightarrow{1/0} s_3 \xrightarrow{0/0} s_1 \xrightarrow{1/1} s_2 \xrightarrow{0/0} s_3 \xrightarrow{1/0} s_0 \xrightarrow{1/0} s_3$ biçimindedir. Çıktı `001000` olur (Bölüm 5).

Soru 2 (derste sözlü, 00:18:25)

Durum diyagramı verilen üç durumlu makine hangi işlemi gerçekleştirmektedir?

Çözüm: Hoca örnek girdi olarak $12 = 1100_2$ verdi. Soldan okununca $12 \to 6 \to 3 \to 1$ elde edildi. Makine sayıyı ikiye böler, yani bir bit sağa kaydırır (Bölüm 7).

Soru 3 (derste sözlü, 00:24:12)

İki tane bir bitlik sayıyı toplayan sonlu durum makinesini tasarlayın.

Çözüm: İki durum: s_0 elde yok, s_1 elde var. Durum tablosu Bölüm 8.4'tedir.

Soru 4 (derste sözlü, 00:30:29)

Gönderilen mesajda art arda üç 1 varsa dışarıya hata mesajı olarak 1 veren bir sonlu durum makinesi tasarlayın.

Çözüm: Üç durum yeterlidir. Her 0'da s_0'a dönülür; $s_0 \xrightarrow{1/0} s_1 \xrightarrow{1/0} s_2$ ve $s_2 \xrightarrow{1/1} s_2$ (Bölüm 9.2).

Soru 5 (derste sözlü, 00:38:45)

Tam 4 karakterli, ilk karakteri harf, diğer karakterleri harf ya da rakam olan değişken adlarını denetleyen bir makine tasarlayın. Geçersiz karakterler (Σ) kabul edilmesin ve yeniden girilsin.

Çözüm: $S_0 \to S_1 \to S_2 \to S_3 \to S_4$ zinciri; S_4 bitiş durumu. Σ kenarları kendi üzerine döner, "baştan başla" denirse S_0'a bağlanır (Bölüm 10.2).

Soru 6 (derste sözlü, 00:45:40)

Sola kaydırma yapan bir makine tasarlayın.

> **Çözüm (derste verilmedi):** Bölüm 7'deki makine bunu zaten yapar; yalnız bitler sağdan (en düşük basamaktan) verilir ve her çıktı biti okunan bitin yerine yazılır. Makine her adımda bir önceki biti verir: S_0 başlangıç; son okunan bit 1 ise S_1'e, 0

ise S_2'ye gidilir; çıktı, bulunulan durumun hatırladığı bittir (S_0 ve S_2'de 0, S_1'de 1). 1100_2 sağdan verilince çıktı $1000_2 = 8$ olur; bu, 4 bitte sola kaydırmadır: $(2 \cdot 12) \bmod 16 = 8$. Python ile bütün 4 bitlik sayılar için $(2n) \bmod 16$ sonucu doğrulandı.

Soru 7 (derste sözlü, 00:45:46)

İki tane iki bitlik sayıyı nasıl toplarsınız?

Çözüm (derste verilmedi): Bölüm 8'deki seri toplayıcı yeterlidir. Bitler en düşük basamaktan başlanarak çiftler hâlinde verilir; elde durum olarak bir sonraki basamağa taşınır. En sonda oluşabilecek elde için bir 00 çifti daha verilir. Örneğin $11_2 + 01_2$: önce $(1,1)$ verilir, $s_0 \xrightarrow{11/0} s_1$; sonra $(1,0)$, $s_1 \xrightarrow{10/0} s_1$; sonra $(0,0)$, $s_1 \xrightarrow{00/1} s_0$. Çıktılar ters sırada okunur: $100_2 = 4 = 3 + 1$. Python ile bütün 4 bitlik sayı çiftleri için doğru toplam elde edildi.

Soru 8 (derste sözlü, 00:45:51)

Çıkarma işlemini yapan makineyi nasıl tasarlarsınız?

Çözüm (derste verilmedi): Hocanın dediği gibi iki durum vardır: s_0 borç alınmadı, s_1 borç alındı. Girdi xy çifti, en düşük basamaktan başlanarak verilir ve $x - y$ hesaplanır. Her adımda $t = x - y - (\text{borç})$ bulunur; $t < 0$ ise borç alınır (s_1), çıktı $t \bmod 2$ olur.

Durum	00	01	10	11
s_0 (borç yok)	s_0, 0	s_1, 1	s_0, 1	s_0, 0
s_1 (borç var)	s_1, 1	s_1, 0	s_0, 0	s_1, 1

Python ile $x \geq y$ olan bütün 4 bitlik sayı çiftleri için doğru fark elde edildi.

Dersin Toparlaması

Bu derste sonlu durum makineleri turnike ve asansör gibi günlük hayat örnekleriyle tanıtıldı, doğal dil işleme ve görüntü işlemedeki kullanımlarına değinildi. Çıktılı sonlu durum makinesi $M = (S, I, O, f, g, s_0)$ altılısıyla tanımlandı. Durum tablosundan diyagram çizmek, diyagramdan tablo yazmak ve bir girdi dizisinin çıktısını izleyerek bulmak gösterildi. Başlangıç durumunun tek, bitiş durumlarının birden çok olabileceği; akan veride bitiş durumu olmadığı, durma noktası belli olduğunda ise bitiş durumu konduğu anlatıldı.

Tasarım tarafında en önemli adımın kaç durum gerektiğine karar vermek olduğu vurgulandı: toplamada elde var/yok, art arda 1 saymada sayılan 1 sayısı, değişken adında girilen karakter sayısı. Sağa kaydırma örneği, girdinin hangi uçtan okunduğunun sonucu değiştirdiğini gösterdi. Ders, ayrık matematiğin lojik, veri yapıları, algoritmalar ve hesaplama kuramı gibi derslere terminoloji ve bakış açısı kazandırdığı vurgusuyla kapandı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.